

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl



Me Presento

- Mi nombre es César Núñez
- Ingeniero Comercial & Master en Finanzas de la Universidad de Chile.
- Master en Business Analytics & Big Data en MIP Politecnico di Milano.
- 6 años de experiencia como Analista de Riesgo Financiero en Bci, McKinsey y Tanner Servicios Financieros.
- Experiencia como profesor en la Universidad de Chile, Universidad Finis Terrae y en la Universidad Santo Tomás.



- El curso tiene como objetivo ser un punto de introducción para la introducción a la programación en Python, manejo de bases de datos, creación de dashboards y automatización de procesos

Unidad de Aprendizaje	Foco de Contenido Sincrónico	Crono.	Pedag.
1. Introducción a Python	Sintaxis, tipos de datos y control de flujos.	1h	2h
2. Manipulación de Dataframes	Lectura/escritura y limpieza.	3h	4h
3. Visualización de datos	Gráficos y Dashboards.	14h	20h
4. Automatización de procesos	Registro de tareas y envío de reportes.	1h	2h
Horas Totales		19h	28h

Evaluaciones

La calificación final del módulo se compone de actividades individuales y un proyecto colaborativo, distribuidos de la siguiente manera:

Evaluación Individual (60 %)

Consiste en **2 evaluaciones** de carácter personal a lo largo del curso:

- **Evaluación 1 (30 %):** Fundamentos de programación, carga de datos y manipulación de Dataframes.
- **Evaluación 2 (30 %):** Lógica de visualización estática e interactiva.

Trabajo en Grupo (40 %)

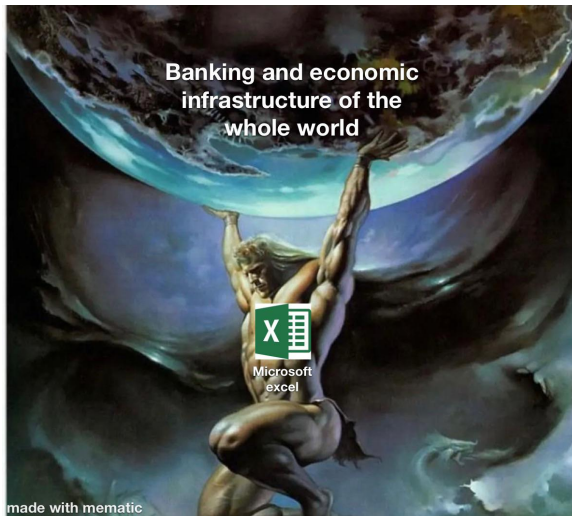
Consiste en el desarrollo colaborativo de un proyecto final aplicado:

- **Proyecto Final (40 %):** Construcción, despliegue y automatización de un **Dashboard interactivo** completo para Control de Gestión.

- No se borran notas, de ningun tipo.
- Si falta a clases, es responsabilidad del estudiante ponerse al día
- Plagios parciales y/o totales se castigan con la nota minima en la evaluacion afectada.

- Las clases seran expositivas y de discusion, el material estara habilitado previo a la realizacion de la clase.
- Si me necesitan contactar, me envían un correo si no contesto en 24hrs me insisten.

Por qué python?



¿Qué es un dato?

- Representación simbólica de un hecho, concepto o instrucción
- Puede ser numérico, textual, de fecha, booleano, etc.
- En contexto empresarial: información que permite tomar decisiones
- Ejemplo básico: “150 unidades vendidas” es un dato

Ejemplos en contextos empresariales

- **Ventas:** Cantidad vendida, precio unitario, fecha de venta
- **Recursos Humanos:** Edad del empleado, salario mensual, años de antigüedad
- **Marketing:** Tasa de conversión, número de clics, origen del lead
- **Inventario:** Stock disponible, costo de adquisición, proveedor
- **Finanzas:** Ingresos totales, gastos operativos, margen de ganancia

Demostración práctica en Excel

- Se abre Microsoft Excel
- Se copia/pega datos crudos desde una fuente externa
- Se aplican las tres etapas clave que veremos a continuación

1. Definición de variables clave

- **Variable:** característica o atributo que se quiere medir o registrar
- Ejemplos en análisis de clientes:
 - Nombre (texto)
 - Edad (numérico)
 - Monto de compra (numérico, moneda)
 - Fecha de compra (fecha)
 - Ciudad (texto / categórica)
- Pregunta clave: ¿Qué información es realmente útil para el análisis?

2. Organización inicial de datos crudos

Datos crudos (desordenados):

- Juan Pérez 28 45000
- 35 Ana Gómez 62000
- Pedro Ruiz,32,51000

Tabla estructurada:

Nombre	Edad	Compra (\$)
Juan Pérez	28	45.000
Ana Gómez	35	62.000
Pedro Ruiz	32	51.000

3. Introducción al Ciclo del Análisis de Datos

- **Organización** → facilita las siguientes fases
- Limpieza: elimina duplicados y errores
- Análisis: cálculos y estadísticas rápidas
- Visualización: gráficos claros y profesionales

Ejercicio individual: Lista de clientes

Instrucciones (5 minutos):

- Recibe la siguiente lista desordenada:
 - María López 24 32000 Santiago
 - Carlos Ruiz, 41, 78000, Valparaíso
 - 29 Ana Torres 45000
- Crea una tabla estructurada en Excel con columnas:
 - Nombre
 - Edad
 - Monto compra (\$)
 - Ciudad
- Guarda el archivo como "Ejercicio_Clientes_TuNombre.xlsx"

Caso práctico: Datos de ventas desordenados

Datos recibidos (mezclados y mal etiquetados): XXX

Producto A 1500 2025-01-15

2800 Cliente X Producto B

2025-02-03 950 Producto C

Tarea grupal/individual:

- Identificar variables clave
- Estandarizar formatos y etiquetas
- Crear tabla organizada

Diferencias entre formatos de archivos en Excel

Formato	Extensión	Filas máximas	Tamaño archivo	Uso recomendado
Excel 97-2003	.xls	65.536	Grande	
Excel moderno	.xlsx	1.048.576	Pequeño	
Excel con macros	.xlsm	1.048.576	Pequeño	
Valores separados	.csv	Ilimitado*	Muy pequeño	
Texto plano	.txt	Ilimitado*	Muy pequeño	

* El límite real depende del programa que abra el archivo (Excel permite más de 1 millón de filas).

Consejo rápido

- Usa **.xlsx** para trabajo normal
- Usa **.xlsm** solo si necesitas macros
- Usa **.csv** para compartir datos con otros sistemas o programas

- **Ecosistema de Trabajo Recomendado:**

- Anaconda Distribution como estándar industrial.
- Uso de Jupyter Notebooks para análisis exploratorio ágil.
- Visual Studio Code para desarrollo de scripts estructurados.

- **Filosofía de Python:** Es un lenguaje interpretado, de tipado dinámico y de sintaxis limpia y legible.

```
# Variables y asignaciones basicas
ingresos = 150000
costos = 90000
```

Tipos de Datos Primitivos y Operaciones

Tipos de Datos Primitivos:

- `int` (Enteros): Cantidades, IDs.
- `float` (Decimales): Tasas, márgenes.
- `str` (Texto): Centros de costo, nombres.
- `bool` (Booleanos): Estados (True/False).

Operaciones Matemáticas:

- Suma (+), Resta (-), Multiplicación (*).
- División (/), División entera (//).
- Módulo (%).
- Potencia o exponente (**).

Estructuras de Datos: Listas y Diccionarios

- **Listas:** Colecciones ordenadas y mutables de elementos accesibles mediante un índice con base cero.

```
meses = ['Ene', 'Feb', 'Mar']  
# Acceso por índice: meses[0] -> devuelve 'Ene'
```

- **Diccionarios:** Estructura mapeada de clave-valor ideal para registros etiquetados o semánticos.

```
kpi = {'nombre': 'EBITDA', 'valor': 25000}  
# Acceso por clave: kpi['nombre'] -> devuelve 'EBITDA'
```

Estructuras de Control: Sentencias Condicionales

Evaluación de condiciones lógicas para la toma de decisiones y bifurcación del flujo del programa basándose en indentación.

```
# Control de flujo con condiciones excluyentes
if margen > 0.2:
    print('Margen Saludable')
elif cumple_porcentaje >= 1.0:
    bono = 100
elif cumple_porcentaje >= 0.9:
    bono = 50
else:
    print('Alerta de Margen')
    bono = 0
```

Bucles e Iteraciones: WHILE y FOR

- **Bucle WHILE:** Repetición de instrucciones basada en una condición de control que se mantiene verdadera.

```
while saldo < meta:  
    saldo += ahorro
```

- **Bucle FOR:** Iteración sistemática y directa sobre una colección, rango de valores o secuencias.

```
for mes in meses:  
    print(mes)
```

Funciones y Librerías Esenciales

- **Funciones:** Bloques de código parametrizados y reutilizables definidos mediante la palabra reservada `def`.

```
def calc_margen(ing, cost):  
    return (ing - cost) / ing
```

- **Librerías Clave en Data Science:**

- pandas: Estructuración y análisis avanzado de datos.
- numpy: Álgebra lineal, matrices y cálculo numérico.
- matplotlib / plotly: Creación de gráficos interactivos.

- **Gestión de Librerías (Instalación e Importación):**

```
# Instalacion mediante terminal: pip install pandas  
import pandas as pd
```

Pandas: El DataFrame y Lectura de Archivos

- **¿Qué es un DataFrame?** La estructura de datos central de Pandas; representa una tabla bidimensional con índices en filas y columnas (homóloga a una hoja de cálculo).
- **Carga Automatizada a Memoria:**

- **Archivos CSV:**

```
df = pd.read_csv('ventas.csv', sep=';')
```

- **Archivos de Excel:**

```
df = pd.read_excel('presupuesto.xlsx', sheet_name='Q1')
```

Exploración Inicial del DataFrame

- **Métodos rápidos para auditar la estructura e integridad de los datos cargados:**
 - `df.head()`: Visualiza las primeras 5 filas de la tabla.
 - `df.info()`: Inspecciona los tipos de datos de cada columna e identifica valores nulos.
 - `df.describe()`: Computa un resumen con métricas estadísticas descriptivas clave.
- **Selección de Variables de Interés (Columnas):**

```
# Seleccion de una unica columna (retorna una Serie)
df['Ingresos']

# Seleccion de multiples columnas (retorna un DataFrame)
df[['Mes', 'Ingresos']]
```

Segmentación y Operaciones Vectorizadas

- **Selección de Filas por Índices:**

- `df.iloc[0:5]`: Segmentación por posición física del registro.
- `df.loc[0:5]`: Segmentación basada en etiquetas del índice.

- **Operaciones Vectorizadas:** Operaciones masivas sobre columnas enteras sin usar bucles manuales.

```
df['Resultado'] = df['Ingresos'] - df['Costos']
```

- **Filtrado Lógico de Datos:** Extracción condicional de filas.

```
df_alto_ingreso = df[df['Ingresos'] > 10000]
```

Exportación de Resultados y Caso Práctico

- **Exportación de Datos a Disco:**

```
df.to_excel('reporte_procesado.xlsx', index=False)
```

- **Caso Práctico de Cierre:** Integración final en un único script funcional.

- ① Lectura de un origen de datos de ventas externo.
- ② Cálculo vectorizado del margen unitario de ganancia.
- ③ Filtrado y aislamiento de anomalías (márgenes negativos).
- ④ Exportación automatizada de alertas críticas de vuelta a Excel.

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

